Advanced Artificial Intelligence and Machine Learning

Pretrained Transformers



Learning goals

- Pretraining objectives: autoregressive (GPT) vs. masked (BERT) language modeling
- GPT-1 to GPT-3: from fine-tuning to zero-/few-shot in-context learning
- Decoding: greedy and beam search, sampling with temperature, top-k, top-p
- BERT and T5: bidirectional encoders and the text-to-text framing
- Instruction tuning, chain-of-thought prompting, and reasoning models (DeepSeek-R1)

From Architecture to Pretrained Models

RECAP: THREE TRANSFORMER FAMILIES

- Session 7 ended with one Transformer, three ways to use it — distinguished only by which positions may attend to which:
  - **Decoder-only** (causal self-attention): generates one token at a time — **GPT**
  - **Encoder-only** (bidirectional self-attention): builds representations of a complete input — **BERT**
  - **Encoder-decoder** (both, plus cross-attention): maps one sequence to another — **T5**
- This session: how each family is *trained* into a useful model, and how the resulting models are *used*.

TODAY: PRETRAIN, THEN ADAPT

- All three model families follow the same recipe:
  1. **Pretrain** on huge amounts of unlabeled text with a self-supervised objective — the text itself provides the labels
  2. **Adapt** to a task: fine-tune the weights, or (for large generative models) just describe the task in the prompt
- The families differ in the pretraining *objective*: predict the next token (GPT), fill in masked tokens (BERT), or rewrite corrupted text (T5).
- Roadmap: language-modeling objectives $\rightarrow$ GPT $\rightarrow$ decoding $\rightarrow$ BERT $\rightarrow$ T5 $\rightarrow$ instruction tuning, chain-of-thought, reasoning models.

Language Modeling Objectives

AGAIN: WHAT IS A LANGUAGE MODEL?

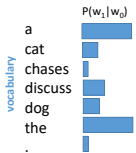
- Statistical model that predicts text that fits well for a given context (typically also text)
- Auto-regressive LMs (ARLMs)
  - Predict one word that is highly likely given a prompt (previous words)
  - For predicting an entire text, repeat the process (i.e., extend the prompt with previously predicted words)
  - To predict a text from scratch, use an extra symbol `<START>` as the initial prompt

ARLM: TOY EXAMPLE

<START>

w_0

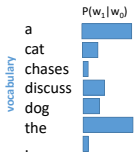
ARLM: TOY EXAMPLE



<START>

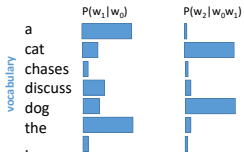
$\mathbf{w_0}$

ARLM: TOY EXAMPLE



<START>	the
w_0	w_1

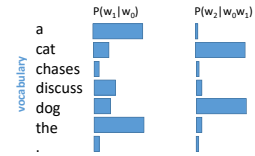
ARLM: TOY EXAMPLE



<START> **the**

w_0 w_1

ARLM: TOY EXAMPLE



<START>	the	dog
w_0	w_1	w_2

ARLM: TOY EXAMPLE

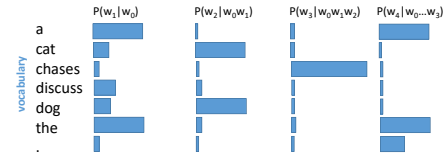


<START>	the	dog
w_0	w_1	w_2

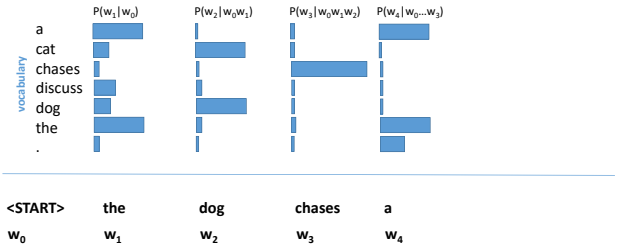
ARLM: TOY EXAMPLE



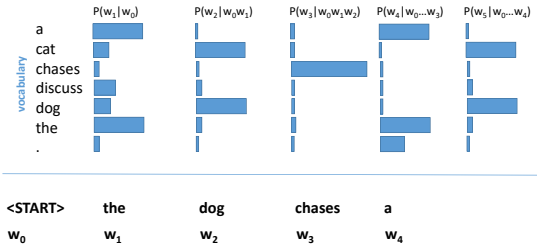
ARLM: TOY EXAMPLE



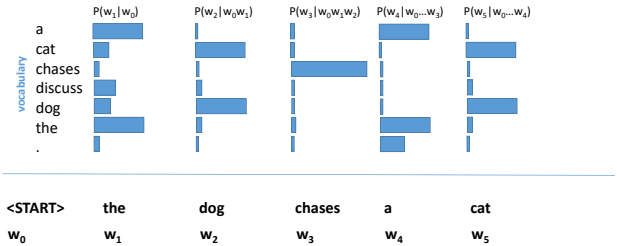
ARLM: TOY EXAMPLE



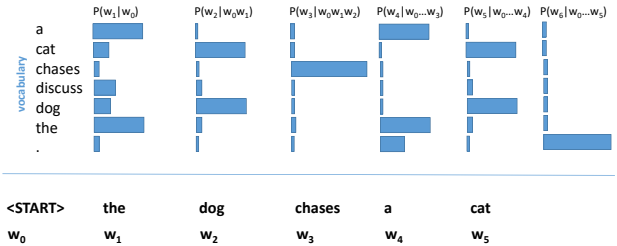
ARLM: TOY EXAMPLE



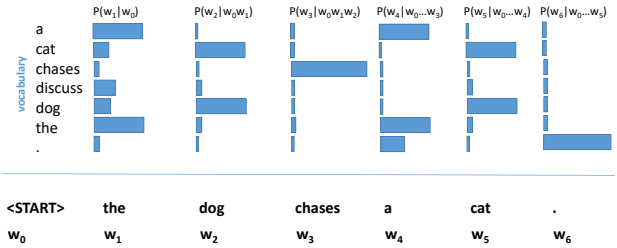
ARLM: TOY EXAMPLE



ARLM: TOY EXAMPLE



ARLM: TOY EXAMPLE



ARLM: PROBABILISTIC INTERPRETATION

- Gives an estimate for the probability of a sentence using conditional probabilities
- In general:

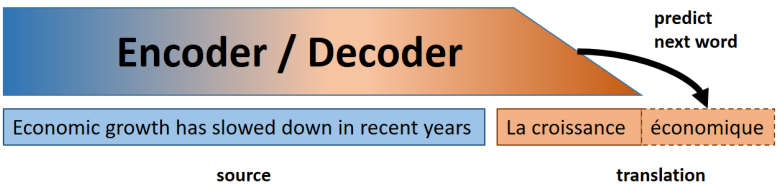
$$P(A \cap B) = P(B) \cdot P(A|B)$$

- For $P(sentence)$:

$$P(w_1, w_2, \ldots, w_n|w_0)$$
$$= P(w_1|w_0) \cdot P(w_2|w_0, w_1) \cdot \ldots \cdot P(w_n|w_0, \ldots, w_{n-1})$$

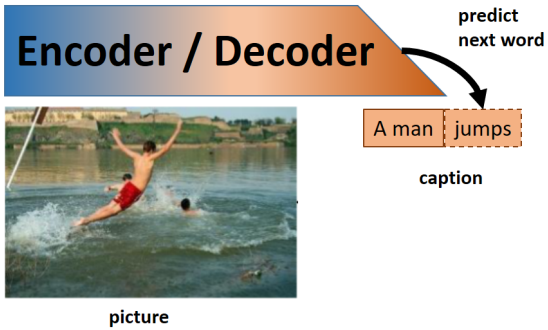
EXAMPLES OF ARLMS (1)

Neural Machine Translation:



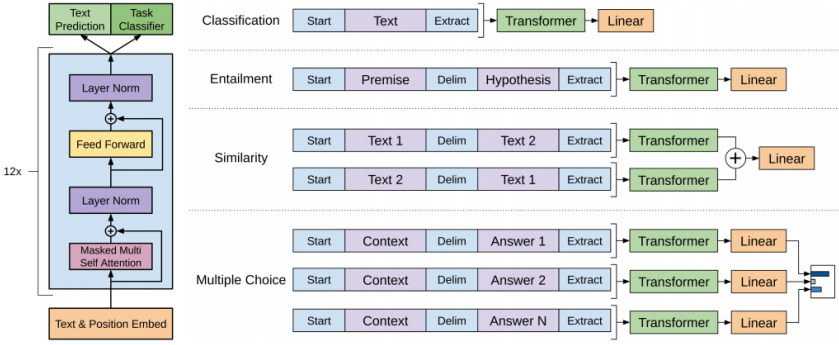
EXAMPLES OF ARLMS (1)

Image Captioning:



GPT-1

GPT-1



▸ Source: Radford et al., 2018

ARCHITECTURAL DETAILS

- Transformer *decoder* as backbone of the architecture
  - 12-layer-decoder with masked self-attention mechanism
  - Hidden dimension $H = 768$, $A = 12$ Attention heads
  - BPE vocabulary w/ 40k merges
  - Learned positional embeddings (as opposed to fixed, sinusoidal ones in the original Transformer)
- With $U = (w_{t-k}, \ldots, w_{t-1})$

$$\vec{h}_0 = \vec{U}\vec{W}_e + \vec{W}_p$$
$$\vec{h}_l = \mathit{Trafo}(\vec{h}_{l-1}) \forall l \in [1, n]$$
$$P(w_t) = \mathit{softmax}(\vec{h}_n \vec{W}_e^\top)$$

PRE-TRAINING GPT

- Standard LM objective

$$L_1(\{w_1, \ldots, w_n\}) = \sum_i \log(P(w_t | w_{t-k}, \ldots, w_{t-1}; \Theta))$$

  where $\{w_1, \ldots, w_n\}$ is an *unlabeled* sequence of tokens

- *Resource:* BooksCorpus
  - $>$ 7k unpublished books from various genres
  - contains long texts and thus allows learning long range dependencies

FINE-TUNING GPT

- Linear output layer with softmax activation on top
- Auxiliary language modeling objective during fine-tuning
  $\rightarrow$ Improves generalization
  $\rightarrow$ Accelerates convergence
- Task-specific input transformations
  - *Entailment:*
    Concatenation of premise (p) & hypothesis (h): $[p; \$; h]$
  - *Similarity:* Use both orderings and concatenate resulting representations: $[s_1; \$; s_2]$ and $[s_2; \$; s_1]$
  - *Q&A and Commensense Reasoning:*
    Concatenate context (z), question (q) and each possible answer (a_k): $[z; q; \$, a_k]$
- Fine-tuning is rather quick, 3 epochs were sufficient

FINE-TUNING GPT

- Additional objective:

$$L_2(\{w_1, \ldots, w_n\}) = \sum_{x,y} \log(P(y|w_1, \ldots, w_n))$$

  where
  - $P(y|w_1, \ldots, w_n) = softmax(h_l^m W_y)$ and
  - $\{w_1, \ldots, w_n\}$ is a *labeled* sequence of tokens
- Combining both objectives:

$$L_3(\{w_1, \ldots, w_n\}) = L_2(\{w_1, \ldots, w_n\}) + \lambda \cdot L_1(\{w_1, \ldots, w_n\})$$

SOTA RESULTS

Performance on different benchmarks:

Table 2: Experimental results on natural language inference tasks, comparing our model with current state-of-the-art methods. 5x indicates an ensemble of 5 models. All datasets use accuracy as the evaluation metric.

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	89.3	-	-	-
CAFE [58] (5x)	80.2	79.0	89.3	-	-	-
Stochastic Answer Network [35] (3x)	80.6	80.1	-	-	-	-
CAFE [58]	78.7	77.9	88.5	83.3		
GenSen [64]	71.4	71.3	-	-	82.3	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	**61.7**
Finetuned Transformer LM (ours)	**82.1**	**81.4**	**89.9**	**88.3**	**88.1**	56.0

Table 3: Results on question answering and commonsense reasoning, comparing our model with current state-of-the-art methods.. 9x means an ensemble of 9 models.

Method	Story Cloze	RACE-m	RACE-h	RACE
val-LS-skip [55]	76.5	-	-	-
Hidden Coherence Model [7]	77.6	-	-	-
Dynamic Fusion Net [67] (9x)	-	55.6	49.4	51.2
BiAttention MRU [59] (9x)	-	60.2	50.3	53.3
Finetuned Transformer LM (ours)	**86.5**	**62.9**	**57.4**	**59.0**

▸ Source: Radford et al., 2018

GPT-2

STARTING WITH A CONTROVERSY

Release Strategy

Due to concerns about large language models being used to generate deceptive, biased, or abusive language at scale, we are only releasing a much smaller version of GPT-2 along with sampling code. We are not releasing the dataset, training code, or GPT-2 model weights. Nearly a year ago we wrote in the OpenAI Charter: "we expect that safety and security concerns will reduce our traditional publishing in the future, while increasing the importance of sharing safety, policy, and standards research," and we see this current work as potentially representing the early beginnings of such concerns, which we expect may grow over time. This decision, as well as our discussion of it, is an experiment: while we are not sure that it is the right decision today, we believe that the AI community will eventually need to tackle the issue of publication norms in a thoughtful way in certain research areas. Other disciplines such as biotechnology and cybersecurity have long had active debates about responsible publication in cases with clear misuse potential, and we hope that our experiment will serve as a case study for more nuanced discussions of model and code release decisions in the AI community.

▸ Source: OpenAI Blog

THE ARCHITECTURE

- Transformer decoder pre-trained on AR language modeling
- Custom web scrape (not publicly available) of all outbound links from Reddit

→ 8M documents / 40GB of text

- Byte-level BPE for tokenization
- Increased context size: 512 → 1024

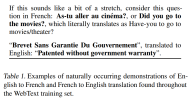
Parameters	Layers	d_{model}
117M	12	768
345M	24	1024
762M	36	1280
1542M	48	1600

▸ Source: Radford et al., 2019

LANGUAGE MODELS AS MULTITASK LEARNERS

"But as exemplified in McCann et al. (2018), language provides a flexible way to specify tasks, inputs, and outputs all as a sequence of symbols.
For example, a translation training example can be written as the sequence (translate to french, english text, french text)."

- Model benefits from seeing "natural" occurrences of task demonstrations during pre-training on large-scale web corpora
  $\rightarrow$ Related rationale to T5 (*literal task descriptions*), but slightly different (*acquired on the fly already during pre-training*).



If this sounds like a bit of a stretch, consider this question in French: **A-t-u allé au cinéma?**, or **Did you go to the movies?**, which literally translates as Have-you to go to movies/theater?

"**Brevet Sans Garantie Du Gouvernement**", translated to English: "**Patented without government warranty**".

Table 1. Examples of naturally occurring demonstrations of English to French and French to English translation found throughout the WebText training set.

▸ Source: Radford et al., 2019

CAPABILITIES – ZERO-SHOT

CAPABILITIES – ZERO-SHOT

- *Zero-shot* refers to solving a task which the model was not previously explicitly trained on, just providing it with a description of the task but no demonstrations, i.e. input/output pairs.
- *Few-shot* would be a relaxation of this setting, allowing for also providing the models with demonstrations (but still no training/gradient updates).
- Radford et al. show GPT-2's zero-shot capabilities on a range of different tasks, paving the way for the developments that came with GPT-3.

CAPABILITIES – ZERO-SHOT

DATASET	METRIC	RESULT	RECORD	HUMAN
Winograd Schema Challenge	accuracy (+)	**70.70%**	63.7%	92%+
LAMBADA	accuracy (+)	**63.24%**	59.23%	95%+
LAMBADA	perplexity (–)	**8.6**	99	~1-2
Children's Book Test Common Nouns (validation accuracy)	accuracy (+)	**93.30%**	85.7%	96%
Children's Book Test Named Entities (validation accuracy)	accuracy (+)	**89.05%**	82.3%	92%
Penn Tree Bank	perplexity (–)	**35.76**	46.54	unknown
WikiText-2	perplexity (–)	**18.34**	39.14	unknown
enwik8	bits per character (–)	**0.93**	0.99	unknown
text8	bits per character (–)	**0.98**	1.08	unknown
WikiText-103	perplexity (–)	**17.48**	18.3	unknown

▸ Source: Radford et al., 2019

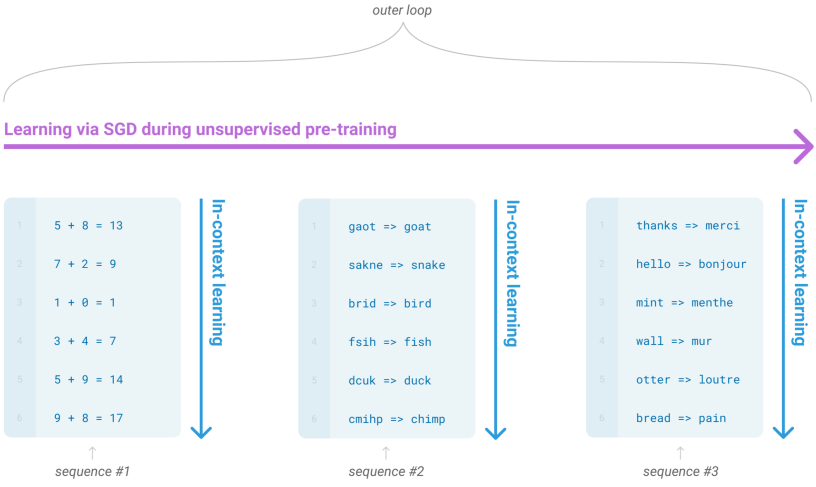
(Note: The numbers correspond to the last row of the Table on the slide "The architecture")

GPT-3 and In-Context Learning

PROMPTING

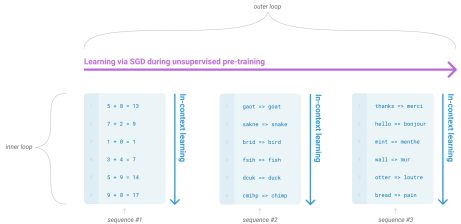
- Salesforce/GPT2: solve tasks by generation through minor reformatting of original tasks (e.g., generate answer in QA, generate label in classification)
- Prompting: solve tasks by generation through carefully designed prompts/instructions that are very different from the original talk description.
- Prompting is usually used within in-context learning (zero-shot, one-shot, few-shot)
- prompting = instruction = task description = pattern

IN-CONTEXT LEARNING



Source: Brown et al., 2020

IN-CONTEXT LEARNING



▸ Source: Brown et al., 2020

- **Pre-training (Outer loop):**
  - Model develops broad set of skills and abilities
  - Trained via gradient descent
- **Inference (Inner loop):**
  - It uses these abilities to rapidly adapt to a desired task (= *in-context learning*)
  - Just a single forward pass w/o any gradient updates

ZERO-SHOT LEARNING

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1  Translate English to French:        <-- task description
2  cheese =>                           <-- prompt
```

▸ Source: Brown et al., 2020

- No gradient updates
- Learning happens “on the fly“
- Model has to “understand“ the task description

ONE-SHOT LEARNING

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← example
3  cheese =>                           ← prompt
```

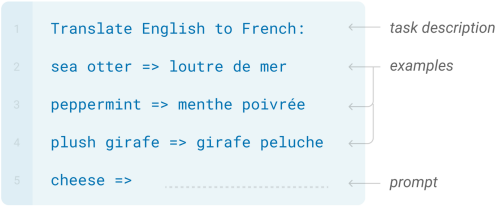
▸ Source: Brown et al., 2020

- No gradient updates
- Model has to "understand" the task description
- Understanding supported by *one* demonstration

FEW-SHOT LEARNING

Few-shot

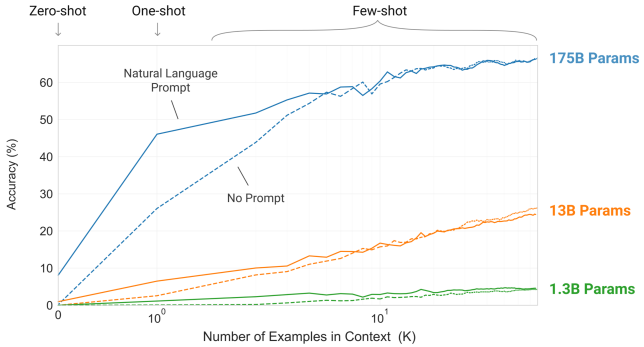
In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



▸ Source: Brown et al., 2020

- No gradient updates
- Model has to "understand" the task description
- Understanding supported by *few* demonstrations

EFFECTIVE IN-CONTEXT LEARNING*



Source: Brown et al., 2020

- Number/Selection of demonstrations = Hyperparameter
- Larger model $\rightarrow$ Better in-context learning capabilities

*on an artificial, simple word scrambling/manipulation task

ARCHITECTURE

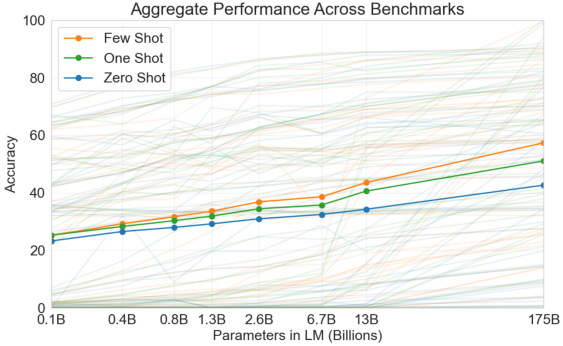
- Various sizes released; GPT-3 usually refers to largest one
- Both width (d_{model}) and depth (n_{layers}) are scaled
- Number of heads adjusted to d_{model}

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

▸ Source: Brown et al., 2020

X-SHOT COMPARISON

- 42 accuracy-denominated benchmarks
- Few-shot performance increases faster than zero-shot



▸ Source: Brown et al., 2020

Decoding

DECODING EXAMPLE (1)

Prompt: Once upon a time

Time step 1:

- Model input: Once upon a time
- Next token: there

Time step 2:

- Model input: Once upon a time there
- Next token: was

Time step 3:

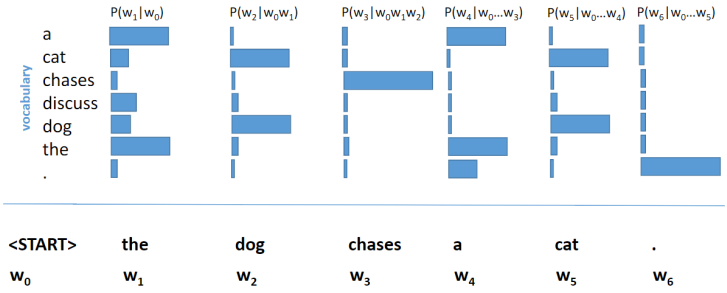
- Model input: Once upon a time there was
- Next token: a

Time step 4:

- Model input: Once upon a time there was a
- Next token: cat

...

DECODING EXAMPLE (2)



- At each timestep the model produces a probability distribution
- A decoding strategy determines how to choose the next token from that distribution, that token is then added to the context
- Generation stops based on stopping criteria (*see: next slide*)

STOPPING CRITERIA FOR TEXT GENERATIONS

- **<EOS> Token**: When this token is generated the model stops
- **Maximum Length**: A predefined maximum length can be set for the generated text. When the text reaches this length, generation stops to prevent excessively long outputs
- **Maximum Time**: A predefined maximum time for generation can be set. After this time has been reached, generation stops
- **Other Criteria**: There are more stopping criteria implemented in huggingface ▸ huggingface

DECODING STRATEGIES

Deterministic

- Greedy search
- Beam search
- Contrastive search ▸ Su et al., 2022
- Contrastive decoding ▸ Li et al., 2023

Stochastic

- Sampling (with temperature)
- Top-k sampling ▸ Fan et al., 2018
- Nucleus top-p sampling ▸ Holtzman et al., 2019
- Typical sampling ▸ Meister et al., 2023

Remark: Other decoding strategies exist, and various combinations are possible, such as top-k sampling with temperature, or top-p sampling followed by top-k sampling (with temperature), etc.

GREEDY SEARCH (1)

- **Core idea**: Greedy search selects the word with the highest probability at each timestep, iteratively building the output sequence
- **Exploration of search space**: It explores a single path through the output space, favoring the most probable word at each step without considering future consequences
- **Candidate Sequence**: Only keeps track of the most likely sequence at each step, discarding other possibilities
- **Decision Making**: It makes local decisions based solely on the highest probability at the current step without considering potential longer-term outcomes

GREEDY SEARCH (2)

- The model accepts an input sequence of tokens $x_1, x_2, ..., x_N$, which we also call prompt
- The model then generates a token at each timestep t until T: $y_1, y_2, ..., y_T$
- In greedy search we choose the token with the highest conditional probability from the vocabulary V
- $y_t = argmax_{y \in V} P(y|y_1, y_2, ..., y_{t-1}, \mathrm{x})$
- With y_t being the chosen token at timestep t and $\mathrm{x} = (x_1, x_2, ..., x_N)$ being the initial prompt

GREEDY SEARCH: EXAMPLE

- Suppose our vocabulary only has four tokens: A, B, C and <eos>

Time step	1	2	3	4
A	0.5	0.1	0.2	0.0
B	0.2	0.4	0.2	0.2
C	0.2	0.3	0.4	0.2
<eos>	0.1	0.2	0.2	0.6

▸ Source: Dive Into Deep Learning Book

- At each timestep greedy search chooses the token with the highest conditional probability
- The model thus predicts A, B, C, <eos>
- Its probability is $0.5 \cdot 0.4 \cdot 0.4 \cdot 0.6 = 0.048$

DRAWBACKS OF GREEDY SEARCH (1)

- Now we select token C at timestep 2 instead of B

Time step	1	2	3	4
A	0.5	0.1	0.1	0.1
B	0.2	0.4	0.6	0.2
C	0.2	0.3	0.2	0.1
<eos>	0.1	0.2	0.1	0.6

▸ Source: Dive Into Deep Learning Book

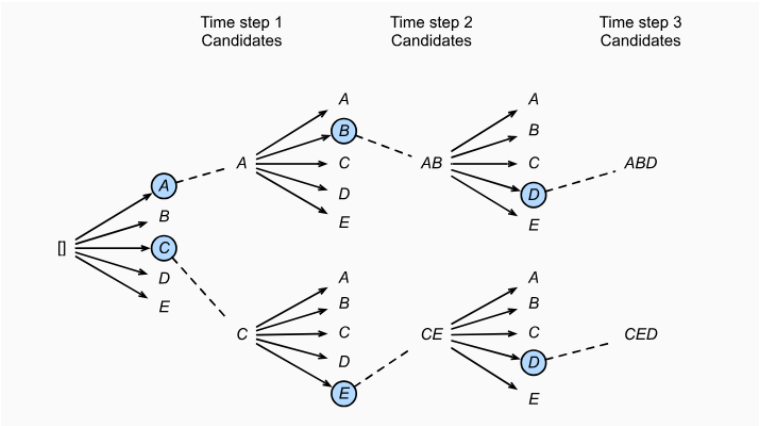
- At the timesteps 3 and 4 the conditional probabilities change since the context is no longer A, B but A, C
- The final token sequence is A, C, B, <eos>
- Its probability is $0.5 \cdot 0.3 \cdot 0.6 \cdot 0.6 = 0.054$
- Even though C at $t = 2$ has a lower probability, the final sequence has a higher probability

BEAM SEARCH

- **Core idea**: Beam search extends the exploration to multiple possible sequences instead of just the most probable one
- **Exploration of search space**: It explores multiple paths (or "beams") simultaneously, maintaining a set of promising candidate sequences
- **Candidate Sequence**: Keeps a fixed number of most probable sequences (determined by the beam width parameter k) at each step
- **Decision Making**: At each step, it considers multiple candidate sequences and selects the most probable ones based on their cumulative probabilities up to that point

BEAM SEARCH: EXAMPLE (1)

- Suppose $V = \{A, B, C, D, E\}$ and beam width $k = 2$



Source: Dive Into Deep Learning Book

BEAM SEARCH: EXAMPLE (2)

- At each timestep beam search chooses the k tokens with the highest joint probability
- Suppose at $t = 1$ A and C have the highest conditional probabilities $P(y_1|\mathrm{x})$
- At $t = 2$ for all $y_2 \in V$ we compute:

$$P(A, y_2|\mathrm{x}) = P(A|\mathrm{x}) \cdot P(y_2|A, \mathrm{x})$$
$$P(C, y_2|\mathrm{x}) = P(C|\mathrm{x}) \cdot P(y_2|C, \mathrm{x})$$

- And we again pick the k sequences with the highest probabilities (AB and CE)
- At $t = 3$ again for all $y_3 \in V$ we compute:

$$P(A, B, y_2|\mathrm{x}) = P(A, B|\mathrm{x}) \cdot P(y_2|A, B, \mathrm{x})$$
$$P(C, E, y_2|\mathrm{x}) = P(C, E|\mathrm{x}) \cdot P(y_2|C, E, \mathrm{x})$$

- We repeat this process unitl the maximum length is reached or unitl the <EOS> token gets generated

SAMPLING MOTIVATION

- *Creativity and Variation*: Sampling methods produce varied outputs for the same input, useful in creative applications like story generation and dialogue systems.
- *Avoiding Repetition*: These methods are less likely to generate repetitive loops compared to deterministic methods.

SAMPLING (WITH TEMPERATURE) (1)

The next token is selected randomly based on its conditional probability distribution. To control the randomness of the output sequence, a temperature parameter can be applied to the softmax function

$$\sigma\left(z_i\right) = \frac{e^{\frac{z_i}{temp}}}{\sum_{j=1}^{N} e^{\frac{z_j}{temp}}}$$

- $temp \to \infty$: Output distribution $\approx$ Uniform distribution
- $temp \to 0$: Output distribution $\approx$ Point mass (Greedy search)

SAMPLING (WITH TEMPERATURE) (2)

Prompt: "Once upon a time"

- Sampling with low temperature: *", during the Second World War, during the final months for his three most talented young players, the coach, Harry Gregg said this"*
- Sampling with high temperature: *"— well. Nowhere you call back my call, not on time; never the two on account my four. Do not come." This old woman — you might have liked, she herself — she did smile."*

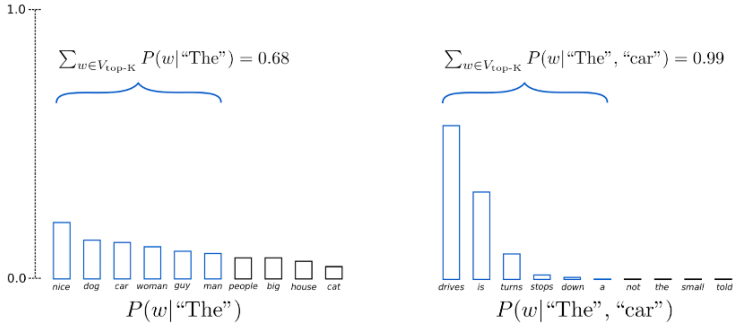
The generated stories are diverse but sometimes very erratic.

$\Rightarrow$ Sample from the top-k tokens

TOP-K SAMPLING

Fan et al., 2018

In Top-k sampling, the k most likely next tokens are filtered, and the probability mass is redistributed. Visualization for $k = 6$ in two sampling steps:



huggingface, Patrick von Platen

TOP-K SAMPLING

Prompt: "Once upon a time"

- Top-k , $k = 100$: *"when I was young the internet was a mysterious landscape full of new and exciting ideas. I read ebooks, watched videos, read short stories"*

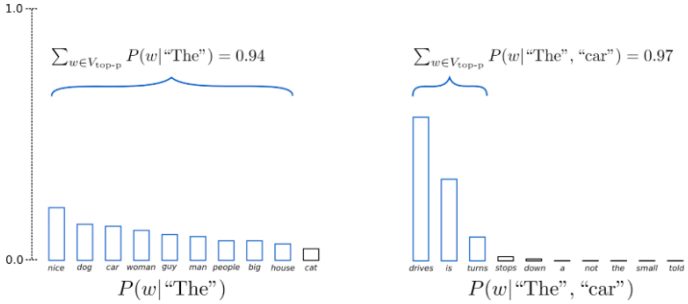
The quality has improved, but the fixed k might be counterproductive

$\Rightarrow$ Make k dynamic

TOP-P (NUCLEUS) SAMPLING

Holtzman et al., 2019

Top-p sampling chooses from the smallest possible set of tokens whose cumulative probability exceeds the probability threshold p. The probability mass is then redistributed accordingly. Visualization with a threshold $p = 0.92$:



huggingface, Patrick von Platen

TOP-P (NUCLEUS) SAMPLING

Prompt: "Once upon a time"

- Top-p , $p = 0.92$: *"there were four major political parties in the United States. Since then, however, they have become even more of a novelty. For the past few decades, there have been only two."*

SOTA for many years, default decoding strategy in various GPT versions, but sometimes erratic depending on p and the sampled tokens.

BERT: Bidirectional Encoders

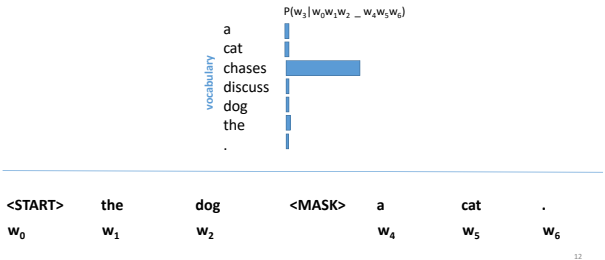
FROM GPT TO BERT

- GPT predicts each token from its *left* context only — the causal mask from session 7 is what makes generation possible.
- For understanding tasks (classification, tagging, retrieval) the whole input is available up front; a token's meaning depends on both sides ("the bank of the river").
- Removing the causal mask gives bidirectional self-attention — but then "predict the next token" becomes trivial, since the model can see it.
- BERT's answer: hide some tokens and predict them from both sides — the **masked language model** objective.

MASKED LANGUAGE MODELS (MLM)

- We have seen auto-regressive LMs
  - context: previous words
  - predict: next word
- Another type: *Masked* LMs (MLMs)
  - context: surrounding words
  - predict: masked word

MLM: TOY EXAMPLE



MLM: PROBABILISTIC INTERPRETATION

- Estimates $P(w_i|w_0, w_1, w_{i-1}, \ldots, w_{i+1}, w_n)$
- No "clean" estimate for $P(\textit{sentence})$, as

$$P(w_1, w_2, \ldots, w_n|w_0)$$

$$\neq P(w_1|w_0, w_2, \ldots, w_n) \cdot P(w_2|w_0, w_1, w_3, \ldots, w_n)$$

- ARLMs are better than MLMs for generating texts
- Advantage of MLMs: Learning contextualized representations

BERT: KEY FACTS

***B**idirectional **E**ncoder **R**epresentations from **T**ransformers:*

- Bidirectionally contextual model
  $\rightarrow$ The embeddings of a single token depend on its left- and on its right-side context (similar to ELMo, but better)
- Completely replaces recurrent architectures by Self-Attention
  $\rightarrow$ parallelizable
- Model can be fine-tuned as a whole

EXCURSUS: ELMO

***E**mbeddings from **L**anguage **Mo**dels* ▸ Peters et al., 2018

- Bidirectional language model (LM)
- Combines a forward LM and a backward LM
- Two-layer biLSTM as main architecture

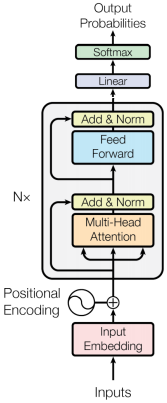
Summary:

- Embeddings are (bidirectionally!) contextualized (as opposed to word2vec)
- Embeddings are *not* adapted to target domain/task (similar as for word2vec)
- Additional weights are learned for each downstream task (i.e. besides the embeddings, no shared knowledge across tasks)

BERT: KEY FACTS

- New self-supervised objective(s)
  - $\rightarrow$ MLM as *necessity* for the architecture to work
  - $\rightarrow$ Next-Sentence-Prediction as complementary objective
- Transformer *encoder* as backbone of the architecture
- 110M (340M) parameters in total for $BERT_{Base}$ ($BERT_{Large}$)
  - 12 (24) Transformer encoder blocks
  - Embedding size of $E = 768$ (1024)
  - Hidden layer size $H = E$
  - $A = H/64 = 12$ (16) attention heads
  - Feed-forward size is set to $4H$

CORE OF BERT – TRANSFORMER ENCODER



▸ Vaswani et al. (2017)

A REMARK ON "CAUSALITY"

Causality is an issue!

- Goal: Learn contextual representations for words/tokens
- *Self-Supervision:* Input and target sequence are the same
  $\rightarrow$ We modify the input to create a meaningful task
- ⚠ Unconstrained Self-Attention makes using the LM objective infeasible
- **Question:** Why is this the case?

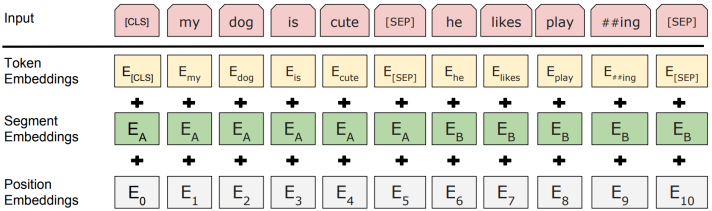
A REMARK ON "CAUSALITY"

Causality is an issue!

- Goal: Learn contextual representations for words/tokens
- *Self-Supervision:* Input and target sequence are the same
  $\rightarrow$ We modify the input to create a meaningful task
- ⚠ Unconstrained Self-Attention makes using the LM objective infeasible
- **Question:** Why is this the case?

  Bidirectionality at a lower layer would allow a word to see itself at later hidden layers
  $\rightarrow$ The model would be allowed to cheat!
  $\rightarrow$ This would not lead to meaningful internal representations

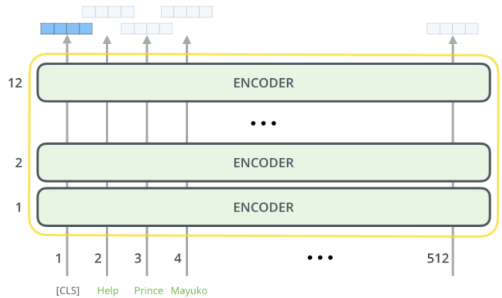
BERT – INPUT EMBEDDINGS



Source: Devlin et al., 2019

- Two concatenated sentences as input
- WordPiece tokenization Wu et al., 2016 for the inputs
  $\rightarrow$ Vocabulary of 30.000 tokens
- *Learned* segment + position embeddings
- Special `[CLS]` and `[SEP]` tokens

BERT – ALL EMBEDDINGS



▸ Source: Jay Alammar

- One embedding per token per layer
- Non-contextual embeddings in the very first embedding layer
- More contextualization deeper into the model

BERT – THE ROLE OF [CLS] AND [SEP]

Why deliberately include extra "words"?

- The `[CLS]` token serves as an overall embedding for representing the whole sequence
  → Later on BERT can thus used for classifying whole sequences
  → Can be extracted and used for clustering or similar
- The `[SEP]` (short for "'separator') token serves as a "signal" for the model when used for taks on pairs of sequences

Note:

- One further "special" token: `[UNK]` for representing unknown tokens or symbols (so the don't "break" the model)

TWO DIFFERENT BERT VERSIONS

There are two different BERT versions, namely BERT base and large. Depending on the architecture we get different parameter counts

▸ Source: Devlin et al., 2019

- **BERT base:**
  - $n_{layers} = 12$, we have 12 encoder layers
  - $n_{heads} = 12$, this will not affect the parameter count since they perfectly split d_{model} across the heads
  - $d_{model} = 768$, thats the embedding dimension
- **BERT large:**
  - $n_{layers} = 24$
  - $n_{heads} = 16$
  - $d_{model} = 1024$

MASKED LANGUAGE MODELING (MLM)

First remark:

- Distinguish from *Masked Self-Attention*
  $\rightarrow$ Masked Self-Attention is an architectural detail in the decoder of the Transformer, i.e. used by e.g. GPT
- Masked Self-Attention as a way to prevent causality issues in a Transformer decoder
- MLM is a self-supervised *modeling objective* introduced to couple Self-Attention and (deep) bidirectionality without violating causality

MLM (2)

- **Training objective:**

  Given a sentence, predict `[MASK]`ed tokens

- **Generation of samples:**

  Randomly replace* a fraction of the words by `[MASK]`

  *Sample 15% of the tokens; replace 80% of them by `[MASK]`, 10% by a random token & leave 10% unchanged

- **Input:**

The	quick	brown	[MASK]	jumps	over	the	[MASK]	dog	.

- **Targets:**

  (*fox*, *lazy*)

MLM (3)

- ⚠ But 20% of the samples will look different!
  → 10% replaced by a random token; 10% left unchanged
- **Input (first case):**

The	quick	brown	`orange`	jumps	over	the	`is`	dog	.

- **Targets (first case):**

(*fox*, *lazy*)

- **Input (second case):**

The	quick	brown	`fox`	jumps	over	the	`lazy`	dog	.

- **Targets (second case):**

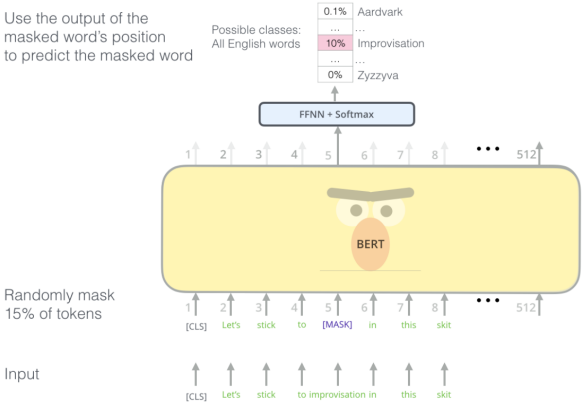
(*fox*, *lazy*)

MLM (4)

Discrepancy between pre-training & fine-tuning:

- `[MASK]`-token as central part of pre-training procedure
- `[MASK]`-token does not occur during fine-tuning
- **Modified pre-training task:**
  Predict 15% of the tokens of which only 80% have been replaced by `[MASK]`
  - 80% of the selected tokens are actually `[MASK]`ed
  - 10% of the selected tokens: The model has to "understand" that the word needs to be *replaced*
  - 10% of the selected tokens: The model has to "understand" that the word needs to be *kept*
- This ensures overlap between the kind data seen during pre-training and during fine-tuning

MLM (5)



Source: Jay Alammar

NEXT SENTENCE PREDICTION (NSP)

- **Training objective:**

  Given two sentences, predict whether s_2 follows s_1

- **Generation of samples:**

  Randomly sample* negative examples

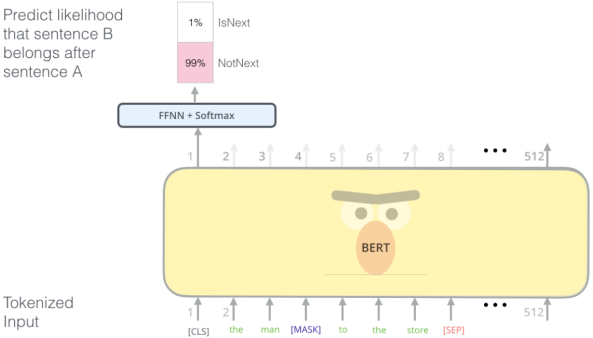
  *50% of the time the second sentence is the actual next sentence, 50% of the time it is a randomly sampled sentence

- **Full Input:**

[CLS]	The	[MASK]	is	quick	.	[SEP]	
It	jumps	over	the	[MASK]	dog	.	[SEP]

- [CLS] token as sequence representation for classification
- [SEP] token for separation of the two input sequences

NSP (2)



▸ Source: Jay Alammar

PRE-TRAINING BERT

Setup:

- 13 GB of text (BooksCorpus + Eng. Wikipedia)
  $\rightarrow$ Back then: **huge**; Now: rather **small**
- Train for approximately* 40 epochs
- 4 (16) Cloud TPUs for 4 days for the BASE (LARGE) variant
- Loss function:

$$Loss_{BERT} = Loss_{MLM} + Loss_{NSP}$$

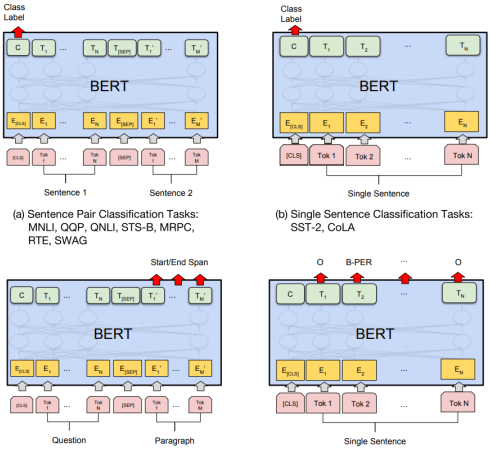
*1.000.000 steps on batches of 256 sequences with a sequence length of 512 tokens

PRE-TRAINING BERT

Sequence lengths:

- For their experiments:
  - Pre-train w/ sequence length 128 for 90% of the steps
  - Pre-train w/ sequence length 512 for 10% of the steps
- *Reason:* Positional embeddings
  - Learned, not sinusoidal (as opposed to the original Transformer)
  - Training on long sequences necessary to learn them well
  - **But:** Training expensive, hence this "compromise"

FINE-TUNING BERT



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG

(b) Single Sentence Classification Tasks: SST-2, CoLA

(c) Question Answering Tasks: SQuAD v1.1

(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

▸ Source: Devlin et al., 2019

FINE-TUNING BERT

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

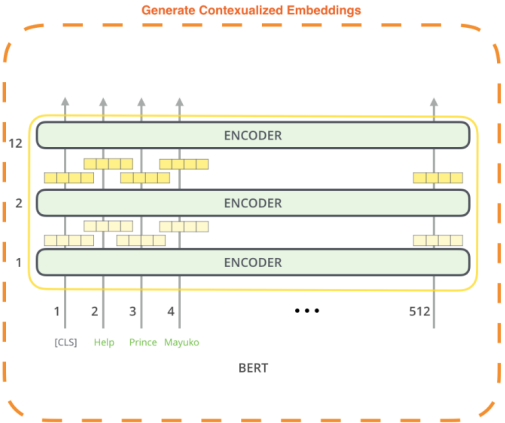
▸ Source: Devlin et al., 2019

- Performance of BERT on the GLUE Benchmark ▸ Wang et al., 2018
- Beats all of the previous state-of-the-art models
- In the meantime: Other models (way) better than BERT

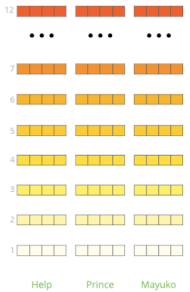
FINE-TUNING DETAILS

- Relatively cheap compared to pre-training:
  - $<$ 1 hour on a single Cloud TPU
  - "a few hours" on a GPU
- Recommendations for hyperparameters:
  - **Batch Size:** 16, 32
  - **Adam learning rate:** 5e-5, 3e-5, 2e-5
  - **#epochs:** 2, 3, 4
  - **Dropout probability:** 0.1
- Data sets w/ $>$ 100k labeled examples rather insensitive to hyperparameters

FEATURE EXTRACTION FROM BERT



The output of each encoder layer along each token's path can be used as a feature representing that token.



But which one should we use?

▸ Source: Jay Alammar

FEATURE EXTRACTION FROM BERT

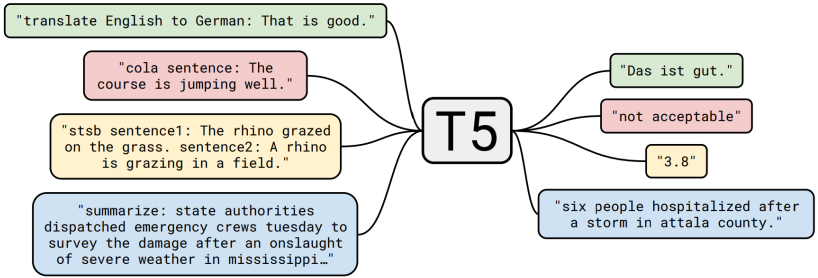
System	Dev F1	Test F1
ELMo (Peters et al., 2018a)	95.7	92.2
CVT (Clark et al., 2018)	-	92.6
CSE (Akbik et al., 2018)	-	**93.1**
Fine-tuning approach		
$\text{BERT}_{\text{LARGE}}$	96.6	92.8
$\text{BERT}_{\text{BASE}}$	96.4	92.4
Feature-based approach ($\text{BERT}_{\text{BASE}}$)		
Embeddings	91.0	-
Second-to-Last Hidden	95.6	-
Last Hidden	94.9	-
Weighted Sum Last Four Hidden	95.9	-
Concat Last Four Hidden	96.1	-
Weighted Sum All 12 Layers	95.5	-

Table 7: CoNLL-2003 Named Entity Recognition results. Hyperparameters were selected using the Dev set. The reported Dev and Test scores are averaged over 5 random restarts using those hyperparameters.

▸ Source: Devlin et al., 2019

T5: Everything Is Text-to-Text

REVISITING TEXT-TO-TEXT TASKS



▸ Source: Raffel et al., 2020

Ingredients:

- `<task prefix>` for each task
  → Concatenation with the input
- Output label / score formatted as string

TEXT-TO-TEXT TRANSFER TRANSFORMER

Short summary:

- In short: T5 (bc of the five Ts)
- A complete encoder-decoder Transformer architecture

  Question: Why is this important?
- Relative positional embeddings
- All tasks reformulated as text-to-text tasks

  $\rightarrow$ Probably the most important innovation of this work
- From BERT-size $\times 2$ (since enc-dec) up to 11B parameters
- Paradigm shift from *Sequential Transfer Learning* towards true *Multi-Task Learning*

▸ Nice Animation (similar to figure before)

INPUT AND OUTPUT FORMAT

Input Side:

- SentencePiece framework w/ WordPiece tokens
- Add task-specific (text) prefix to the original input
- Choice of the prefix: Hyperparameter!!

→ Changing this had limited impact

→ No extensive experiments performed by the authors

Output Side:

- Output as a word or a piece of text (also numerical similarity scores)
- If output not present in set of potential alternatives, prediction considered as wrong (many other possible ways; this is not trivial)

TASK PREFIX VS. PROMPTING

Adding task-specific (text) prefix:

- Add task-specific (text) prefix to the original input
- Model is fine-tuned on samples prepended with this prefix

→ It learns to recognize what it is expected to do when encountering a specific prefix at test time

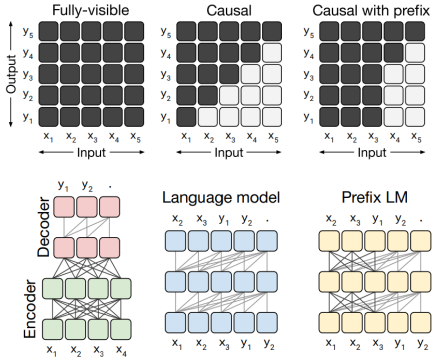
→ Probably because of this: limited impact found by the authors

Prompting:

- Refers to just querying a trained w/o fine-tuning it
- Paradigm of Few-/Zero-Shot Learning
- This is found to have a *huge* impact on model performance

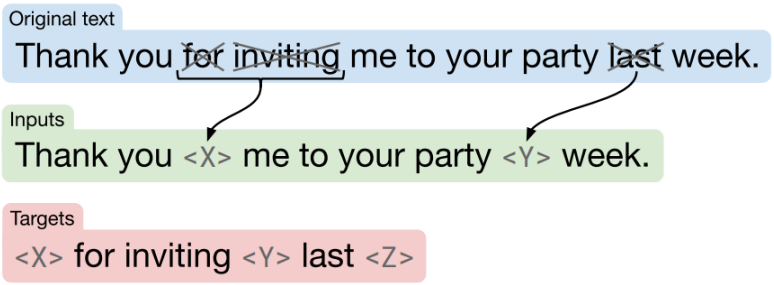
ARCHITECTURAL DIFFERENCES

Attention patterns (top) and Schematics of considered architectures (bottom)



▸ Source: Raffel et al., 2020

PRE-TRAINING OBJECTIVE



▸ Source: Raffel et al., 2020

1. Mark spans in input sequence for corruptions
2. Replace tokens with sentinel tokens
3. Predict sentinel tokens and replaced tokens

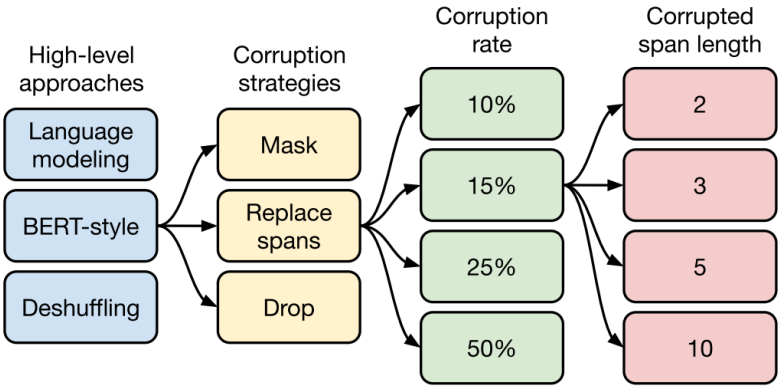
PRE-TRAINING OBJECTIVES

- Authors experimented with different objectives
- Most of them rely in some way on MLM

Objective	Inputs	Targets
Prefix language modeling	Thank you for inviting	me to your party last week .
BERT-style Devlin et al. (2018)	Thank you <M> <M> me to your party apple week .	*(original text)*
Deshuffling	party me for your to . last fun you inviting week Thank	*(original text)*
MASS-style Song et al. (2019)	Thank you <M> <M> me to your party <M> week .	*(original text)*
I.i.d. noise, replace spans	Thank you <X> me to your party <Y> week .	<X> for inviting <Y> last <Z>
I.i.d. noise, drop tokens	Thank you me to your party week .	for inviting last
Random spans	Thank you <X> to <Y> week .	<X> for inviting me <Y> your party last <Z>

▸ Source: Raffel et al., 2020

PRE-TRAINING OBJECTIVES



▸ Source: Raffel et al., 2020

THE COLOSSAL CLEAN CRAWLED CORPUS (C4)

- Effort to measure the effect of quality, characteristics & size of the pre-training resources
- Common Crawl as basis, careful cleaning and filtering for English language
- Orders of magnitude larger (750GB) compared to commonly used corpora

THE COLOSSAL CLEAN CRAWLED CORPUS (C4)

Experiments (with respect to C4)

Data set	Size	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ C4	745GB	83.28	**19.24**	80.88	71.36	**26.98**	**39.82**	**27.65**
C4, unfiltered	6.1TB	81.46	19.14	78.78	68.04	26.55	39.34	27.21
RealNews-like	35GB	**83.83**	**19.23**	80.39	72.38	**26.75**	**39.90**	**27.48**
WebText-like	17GB	**84.03**	**19.31**	**81.42**	71.40	**26.80**	**39.74**	**27.59**
Wikipedia	16GB	81.85	**19.31**	81.29	68.01	**26.94**	39.69	**27.67**
Wikipedia + TBC	20GB	83.65	**19.28**	**82.08**	**73.24**	**26.77**	39.63	**27.57**

Number of tokens	Repeats	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Full data set	0	**83.28**	**19.24**	**80.88**	**71.36**	**26.98**	**39.82**	**27.65**
2^{29}	64	**82.87**	**19.19**	**80.97**	**72.03**	**26.83**	**39.74**	**27.63**
2^{27}	256	82.62	**19.20**	79.78	69.97	**27.02**	**39.71**	27.33
2^{25}	1,024	79.55	18.57	76.27	64.76	26.38	39.56	26.80
2^{23}	4,096	76.34	18.33	70.92	59.29	26.37	38.84	25.81

▸ Source: Raffel et al., 2020

T5 - EXHAUSTIVE EXPERIMENTS

Performed experiments with respect to ..

- *.. architecture, size & objective*

→ enc, dec, enc-dec

→ Between 60M and 11B parameters

- *.. details of the Denoising objective (which was found to work best)*
- *.. fine-tuning methods*

→ Adapter layers

→ Gradual Unfreezing

- *.. Multi-task learning strategies*

→ Examples-proportional mixing

→ Temperature-scaled mixing

DIFFERENT T5 VERSIONS

Similar to BERT T5 has different versions, which all have a different parameter count ▸ Raffel et al., 2020

Model	Parameters	n_{layers}	d_{model}	d_{FFN}	d_{qkv}	n_{heads}
small	60M	6	512	2048	64	8
base	220M	12	768	3072	64	12
large	770M	24	1024	4096	64	16
3B	3B	24	1024	16384	128	32
11B	11B	24	1024	65536	128	128

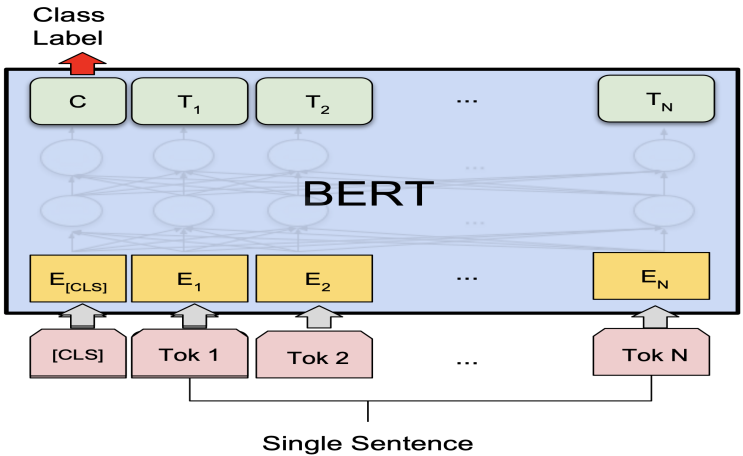
- For T5 small, base and large we have the following parameter count per encoder/decoder layer:
  - $N_{Encoder} = 12 \cdot d_{model}^2$
  - $N_{Decoder} = 16 \cdot d_{model}^2$
- *For T5 3B and 11B $N_{Encoder}$ and $N_{Decoder}$ are different!*

Fine-Tuning, Prompting, Instruction Tuning

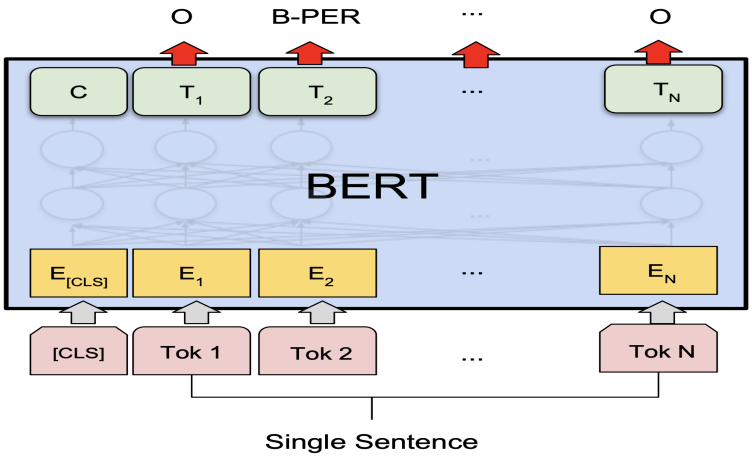
HOW TO SOLVE A TASK WITH LANGUAGE MODELS? (1)

- "old-style" single-task fine-tuning
  - supervised training of the pretrained model on a task-specific training set of size k (where k is not small, e.g., $k = 100$)

BERT FINETUNING EXAMPLE 1



BERT FINETUNING EXAMPLE 2



HOW TO SOLVE A TASK WITH LANGUAGE MODELS? (1)

- "old-style" single-task fine-tuning
  - supervised training of the pretrained model on a task-specific training set of size k (where k is not small, e.g., $k = 100$)
  - the output can be an arbitrary category (e.g., "0", "1" for sentiment analysis)
  - (this is the typical way encoder models like BERT are used)
  - alternatively, the output can be a meaningful "verbalizer" (e.g., "negative", "positive" for sentiment analysis)
    ▸ Schick et al., 2020
  - still very much relevant if you need to deploy a small efficient model for a focused task

HOW TO SOLVE A TASK WITH LANGUAGE MODELS? (2)

- few-shot prompting
  - provide, in-context, k (where k is small, e.g., $k = 5$) examples of what the model is supposed to do
  - the model will then often complete the task just based on analogy to these few shots
  - this is a typical way of using current autoregressive models for tasks
  - no change to the parameters of the model, i.e., no training

FEW-SHOT PROMPTING EXAMPLE

```
1  Translate English to French:
2  sea otter => loutre de mer
3  peppermint => menthe poivrée
4  plush girafe => girafe peluche
5  cheese =>
```

(from gpt3 paper – "Translate English to French" should appear four times to make it a 3-shot example that has 3 identical shots as required by the definition of few-shot prompting)

HOW TO SOLVE A TASK WITH LANGUAGE MODELS? (3)

- multi-task finetuning
  - finetuning on a large training set of *many* tasks
  - the more tasks the better
  - method 1: consistent task format, e.g., each task is reformulated into a question-answering format
  - method 2: more open task format
  - ideally: task format includes "instructions", similar to verbalizer
  - Examples: T0, T5 and FLAN, see below

ISSUES WITH SINGLE-TASK FINE-TUNING

- The result is a single-task model.
  - Generalization of the model is only w.r.t. to one task / data distribution
- Requires annotated data (e.g., $k = 100$)
- **Question:** Could there be other tasks that might benefit?

ISSUES WITH FEW-SHOT PROMPTING: (1) CLASSIFICATION

- Assumption: Model has learned about the task during (unsupervised) pre-training
- Write prompt so that a direct response must be given by the language model.
  - Just a label, just yes/no answer, just a name in QA
- This is not natural dialogic behavior: humans typically don't just answer with a label, yes/no, a name (although sometimes they do)
- Current use of the word "prompt" is more general: basically anything you type into the LLM context window

ISSUES WITH FEW-SHOT PROMPTING: (2) TEXT GENERATION

- Again, assumption: Model has learned about the task during (unsupervised) pre-training
- Again: Write prompt so that a direct response must be given by the language model.
- Works best if the task has occurred during unsupervised training.
- **Question:** For which tasks is this expected to work well?

MULTI-TASK LEARNING: FLAN/T0/T5

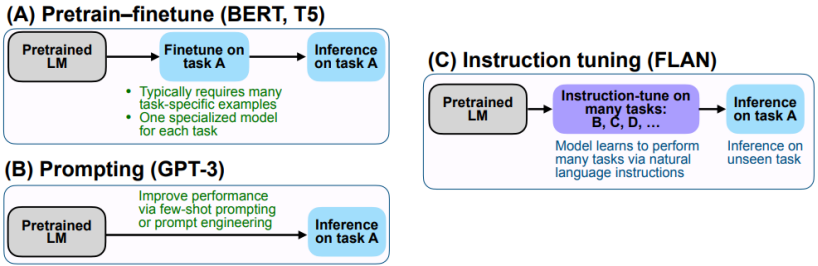


Figure 2: Comparing instruction tuning with pretrain–finetune and prompting.

▸ Source: Wei et al., 2021

MULTITASK PROMPTED TRAINING

- Benchmark for Evaluation:
  - Instead of using held-out *samples* (as is standard in NLP) ...
  - ... we now use held-out *tasks*
  - All data sets belonging to a held-out task go to the test set
  - Generalization across tasks
- The "prompts" are key here to facilitate transfer to new tasks: the model can solve new tasks by generalizing from the train prompts and the ability to generate text outputs.

MULTI-TASK TRAINING VS INSTRUCTION TUNING

- The term "instruction tuning" was introduced by FLAN paper,
  ▸ Source: Wei et al., 2021
- However, today "instruction tuning" refers to a wide variety of methods that train on instruction-output pairs, not just tasks
- In particular, open-ended generation and "alignment" is now included under the rubric "instruction tuning": brainstorming, writing a joke, give me ten analogies for concept X, don't use hate language, don't take positions on controversial political issues etc
- These are not classical NLP tasks.
- Our use of terminology in this class:

MULTI-TASK TRAINING VS INSTRUCTION TUNING / 2

- multi-task finetuning = tasks in the classical NLP sense
- instruction-tuning: a broad approach to changing the behavior of a raw language model (i.e., trained on next-word prediction) to be "helpful, harmless and honest". This includes training on classical NLP tasks, but it's just one part.

FINE-TUNING CONCLUSIONS

- The more training tasks the better
- Multi-task finetuning generalizes across models
  - It works well on different architectures
- Greatly improves usability
- It is relatively compute-efficient
  - Pretraining is hugely expensive, finetuning and instruction tuning are usually much cheaper.
  - For PaLM 540 B it takes 0.2 % of pre-training compute, but improves by 9.4 %

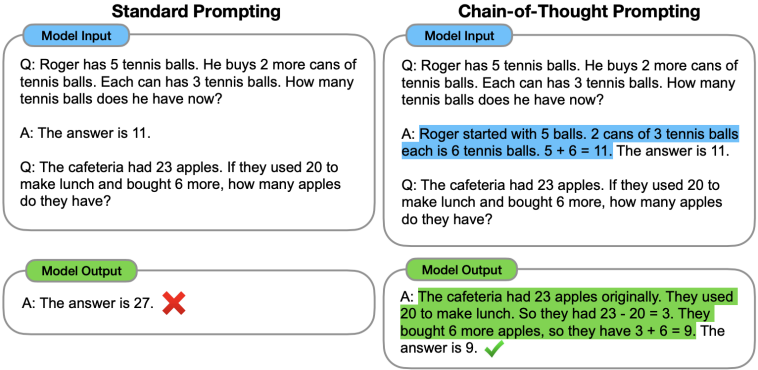
Chain-of-Thought Prompting

Chain-of-thought few-shot prompting MOTIVATION

How to boost the reasoning capabilities of LMs? ▸ Wei et al., 2021

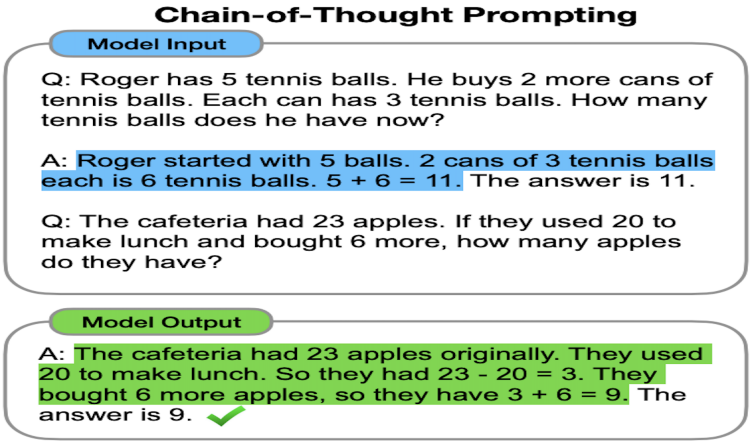
- Use formal approaches, e.g., logic, symbolic reasoning
  - Example: ▸ BeliefBank
  - Difficult to train and deploy, not widely used
- Standard few-shot learning via prompting works for many tasks
  - Still, it works poorly for many tasks that require reasoning
- COT-FS-P
  - A form of few-shot prompting
  - Each "training example" has the form:
    <input, *chain of thought*, output>
  - chain of thought:
    series of reasoning steps that lead to the final answer
  - application: reasoning tasks: complex, commonsense, symbolic etc

Chain-of-thought few-shot prompting PARADIGM



▸ Source: Wei et al., 2022

chain-of-thought few-shot prompting PARADIGM



BENEFITS OF chain-of-thought few-shot prompting

- Decompose multi-step problems and thus allocate more compute to problems requiring more reasoning steps.
- This helps LMs solve problems they otherwise would not be able to solve.
- By describing the reasoning, interpretability is increased. It provides the possibility to observe where reasoning went wrong
- COT-FS-P is closer to how humans solve tasks using language

chain-of-thought few-shot prompting

Following slides: Examples of <input, chain of thought, output> triples for commonsense and symbolic reasoning

▸ Source: Wei et al., 2022

The few shots (training examples) are omitted from these examples to save space, but this is chain-of-thought few-shot prompting, that is, the model is prompted with examples of chain of thought reasoning.

EXAMPLES

StrategyQA

Q: Yes or no: Would a pear sink in water?

A: The density of a pear is about 0.6 g/cm^3, which is less than water. Thus, a pear would float. So the answer is no.

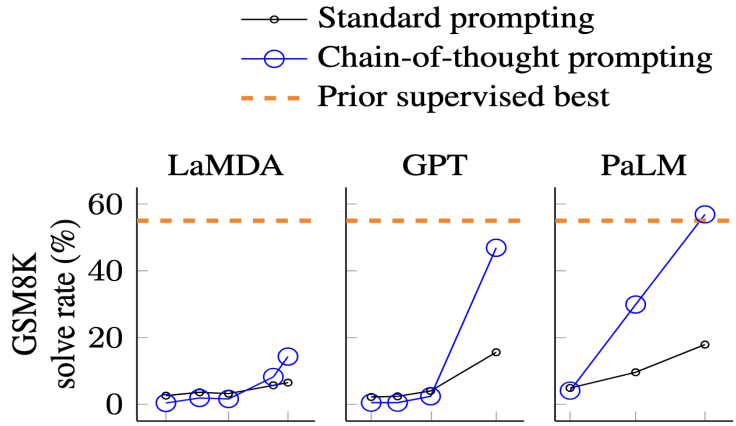
EXAMPLES

Date Understanding

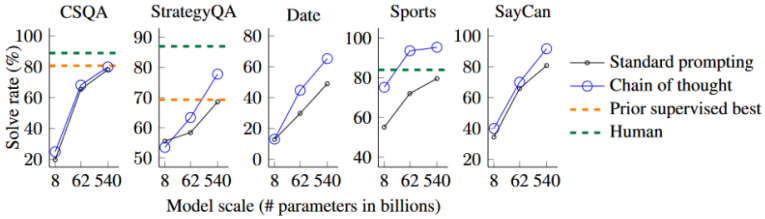
Q: The concert was scheduled to be on 06/01/1943, but was delayed by one day to today. What is the date 10 days ago in MM/DD/YYYY?

A: One day after 06/01/1943 is 06/02/1943, so today is 06/02/1943. 10 days before today is 05/23/1943. So the answer is 05/23/1943.

COT-FS-P IMPROVES ARITHMETIC



COT-FS-P IMPROVES COMMONSENSE



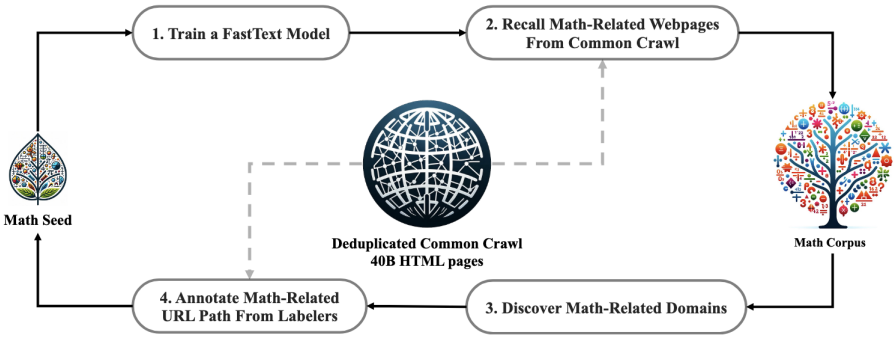
▸ Source: Wei et al., 2022

Reasoning Models: DeepSeek-R1

SOURCES

- First major open reasoning LM (as opposed to proprietary closed): DeepSeek
- ▸ DeepSeek-AI, 2025
- ▸ Nature, 2025

DATA FOR SFT: 40B TOKENS!



▸ DeepSeekMath, 2024

REINFORCEMENT LEARNING

- In reinforcement learning, an agent takes a sequence of actions towards a goal and gets feedback on the success of that sequence at the end.
- For reasoning trajectories, the feedback is: correct or incorrect.
- (thanks to Emma Brunskill)

	supervised learning	reinforcement learning
goal	optimal predictor	optimal policy
learn from experience?	yes	yes
generalization?	yes	yes
feedback	rich (each token)	sparse (in/correct)
feedback when?	immediately	delayed
exploration?	no	yes

REASONING TRACES: FORMAT

- The User asks a question and the Assistant solves it.
- The Assistant first thinks about the reasoning process "in its mind" and then provides the answer to the User.
- The reasoning process and answer are enclosed within <think>...</think> and <answer>...</answer> tags
- `<think> reasoning process here </think>`
- `<answer> answer here </answer>`
- "We intentionally limit our constraints to this structural format, avoiding any content-specific biases to ensure that we can accurately observe the natural progression of the model during the RL process."
- So this is not a few-shot approach!

EXAMPLE OF A REASONING TRACE

Query

The denominator of a fraction is 7 less than 3 times the numerator. If the fraction is equivalent to 2/5, what is the numerator?

Correct reasoning trace

Step 1	*Let the numerator be x.*
Step 2	*The denominator is 3x-7.*
Step 3	*We know that x/(3x-7) = 2/5.*
Step 4	*Therefore, 5x = 6x-14.*
Step 5	*Finally, we get x= 14.* (Correct)

Question:
Is this chain of thought few-shot prompting?

AUTOMATIC, NON-HUMAN REWARDS FOR LM-GENERATED REASONING TRACES

Accuracy rewards evaluate whether the response is correct. For example, in the case of maths problems with deterministic results, the model is required to provide the final answer in a specified format (for example, within a box), enabling reliable rule-based verification of correctness. Similarly, for code competition prompts, a compiler can be used to evaluate the responses of the model against a suite of predefined test cases, thereby generating objective feedback on correctness.

AUTOMATIC VERIFIER/VERIFIABLE REWARDS: AIME EXAMPLE

PROMPT

Let $b \geq 2$ be an integer. Call a positive integer n b-*eautiful* if it has exactly two digits when expressed in base b, and these two digits sum to $\sqrt{n}$. For example, 81 is 13-eautiful because $81 = \underline{6}\ \underline{3}_{13}$ and $6 + 3 = \sqrt{81}$. Find the least integer $b \geq 2$ for which there are more than ten b-eautiful integers.

Please reason step by step, and put your final answer within \boxed{}.

Evaluation

Parse the final answer within \boxed{} and use a rule-based grader to determine if it equals the ground truth. Round numerical values as needed, and use 'SymPy' to parse expressions.

AHA MOMENT

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$

$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$

Rearrange to isolate the inner square root term:

$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$

$\cdots$

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$

We started with the equation:

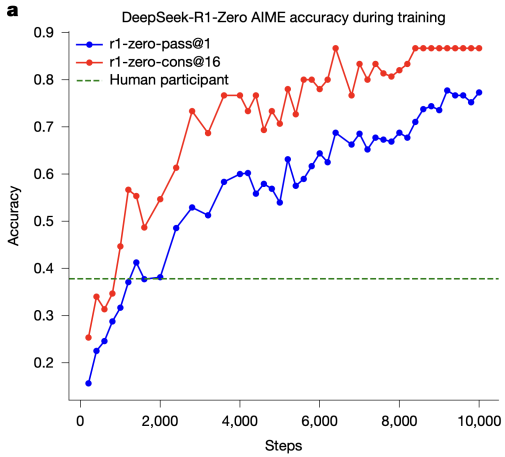
$\sqrt{a - \sqrt{a + x}} = x$

First, let's square both sides:

$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$

Next, I could square both sides again, treating the equation: $\cdots$

ACCURACY AIME: ABOVE HUMAN AVERAGE



UNFAITHFULNESS OF LMS TO TRUE REASONING

```
Human: Answer in one word. What is 36+59?

Assistant: 95

Human: Briefly, how did you get that?

Assistant: I added the ones (6+9=15), carried the 1, then added the tens
(3+5+1=9), resulting in 95.
```

UNFAITHFULNESS OF LMS TO TRUE REASONING

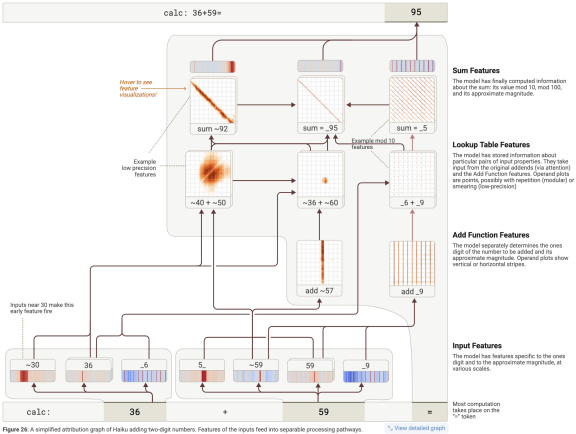


Figure 26. A simplified attribution graph of Haiku adding two-digit numbers. Features of the inputs feed into separable processing pathways.

Where This Is Going

WHERE THIS IS GOING

- Session 9 looks inside the Transformer block: what has changed since 2017 (normalization, activations, position embeddings, attention variants) and mixture-of-experts models such as DeepSeek-V3.
- Session 10 shows that nothing in this session is specific to text: the same encoder-only, decoder-only, and encoder-decoder patterns, and the same pretrain-then-adapt recipe, drive vision and multimodal models.
- DeepSeek-R1's training loop (rewards from automatic verifiers instead of labels) is reinforcement learning — the subject of session 11.

License & Attribution

LICENSE & ATTRIBUTION

- **Main source.** All sections except those listed below are adapted from **Deep Learning for NLP (DL4NLP)**, SLDS, LMU Munich, chapters 4 (*BERT*), 6 (*Post-BERT Era*), 7 (*Generative Pre-Trained Transformers*), 8 (*Decoding Strategies*) and 9 (*Large Language Models*) — licensed **CC BY 4.0**: `https://creativecommons.org/licenses/by/4.0/`. Frames were reordered (GPT first) and selected; the few edits inside frames are listed in `order.txt`. The "Excursus: ELMo" frame takes its bullets from DL4NLP chapter 2 (*Deep Learning Basics*, file "ELMo").
- Figures are reproduced as they appear in DL4NLP, with the attributions DL4NLP itself gives.
- **Original to this course.** "Recap: Three Transformer Families", "Today: Pretrain, Then Adapt", "From GPT to BERT" and "Where This Is Going".
- The two earlier, longer decks this session was merged from (BERT & T5; LLMs) remain in `_previous/`.